

**False Data Injection Detection in IEC 60870-5-104 Smart Grids:
A Comparative Study of Classical Machine Learning Classifiers
Against Conformal Neural Networks**

A PROJECT REPORT

submitted by

CB.EN.U4ELC24028 R ADHITHYA

CB.EN.U4ELC24029 RAJA THOSHAN

CB.EN.U4ELC24054 A K HEMANTH

in partial fulfillment of

BACHELOR OF TECHNOLOGY

IN

ELECTRICAL AND COMPUTER ENGINEERING



AMRITA SCHOOL OF ENGINEERING, COIMBATORE

AMRITA VISHWA VIDYAPEETHAM

COIMBATORE- 641112

March 2026

AMRITA SCHOOL OF ENGINEERING, COIMBATORE
AMRITA VISHWA VIDYAPEETHAM
COIMBATORE CAMPUS



BONAFIDE CERTIFICATE

This is to certify that the project entitled “Cybersecurity-integrated smart grid model using AI algorithms for real-time intrusion detection and power flow optimization” submitted by

CB.EN.U4ELC24028	R ADHITHYA
CB.EN.U4ELC24029	RAJA THOSHAN
CB.EN.U4ELC24054	A K HEMANTH

in partial fulfillment of the requirements for the subject **23ELC212- MACHINE LEARNING** in **Bachelor of Technology** in **ELECTRICAL & COMPUTER ENGINEERING** is a bonafide record of the work carried out under my guidance and supervision at the Department of Electrical and Electronics Engineering, Amrita School of Engineering, Amrita Vishwa Vidyapeetham, Coimbatore Campus.

Mr. Nakkeeran M

Faculty In-Charge, Assistant Professor (OC),
Department of Electrical and Electronics Engineering,
Amrita Vishwa Vidyapeetham, Coimbatore Campus, India.

This project report was evaluated by us on.....(date)

INTERNAL EXAMINER

EXTERNAL EXAMINER

Acknowledgement

We express our sincere gratitude to Mr. Nakkeeran M, Assistant Professor (OC), Department of Electrical and Electronics Engineering, Amrita School of Engineering, Amrita Vishwa Vidyapeetham, Coimbatore, for his invaluable guidance, consistent encouragement, and constructive feedback throughout the course of this project.

We thank the Department of Electrical and Electronics Engineering and Amrita School of Engineering for providing the academic infrastructure, computational resources on Visual Studio Code, and the opportunity to undertake this research under the 23ELC212 Machine Learning curriculum.

We acknowledge the creators of the IEC 60870-5-104 Intrusion Detection Dataset published on Zenodo by Radoglou-Grammatikis et al., whose openly available dataset formed the empirical foundation of this study. We also acknowledge the base paper by Zheng et al. (2025) published in the Cybersecurity journal (DOI: 10.1186/s42400-024-00326-5), which served as our benchmark.

Finally, we thank our families and peers for their constant support and encouragement throughout this project.

Contents

Acknowledgements	iii
List of Figures	vi
List of Tables	vii
Abbreviations	viii
List of Symbols	ix
Abstract	x
1 Introduction	1
1.1 Background and Motivation	1
1.2 Problem Statement	1
1.3 Objectives	2
1.4 Research Gap	3
1.5 Contributions and Paper Organisation	3
2 Related Works	5
2.1 Machine Learning for Intrusion Detection	5
2.2 ICS and SCADA Cybersecurity	5
2.3 Ensemble Methods and Conformal Prediction	5
2.4 Gap Analysis	6
3 Proposed Approach	7
3.1 Process Flow Diagram	7
3.2 Dataset Details	8
3.3 Research Approach	9
4 Methodology	10
4.1 Techniques and algorithms	10
4.2 Evaluation Metrics	13
5 Implementation Details and Results	15
5.1 Experimental Setup (Visual Studio Code)	15
5.2 Dataset Preprocessing	15
5.3 Hyperparameter Tuning Procedure	16

5.4 Experiment 1 - Binary Classification	17
5.5 Experiment 2 - Grouped 3-Class Classification	18
5.6 Experiment 3 - Full 12-Class Classification	19
5.7 Model Fit Analysis: Best Fit, Overfit, Underfit	20
5.8 Cross-Experiment Comparison	23
6 Result Discussion	24
6.1 Insights from Results	24
6.2 Practical Implications	25
6.3 Limitations	25
7 Conclusion and Future Works	27
7.1 Summary of Main Findings	27
7.2 Key Contributions	27
7.3 Key Takeaways	28
7.4 Improvements to Current Work	28
7.5 Extensions and Future Research Directions	28
7.6 Challenges for Future Researchers	29
References	30
Contribution to Sustainable Development Goals	32

List of Figures

5.1	Binary Classification - All Four Metrics with Paper C-NN Baseline	17
5.2	Grouped 3-Class - Test Accuracy, F1 Macro, CV F1 (Stratified K-Fold)	19
5.3	Full 12-Class - Test Accuracy, F1 Macro, CV F1 with Random-Chance Baseline	20
5.4	Test Accuracy Across All Three Experiments	23
5.5	Overfitting Diagnostic - CV F1 vs Test F1 Scatter Plots	21
5.6	Model Fit Status Heatmap - Best Fit / Overfit / Underfit per Task	22

List of Tables

3.1	Dataset Summary - IEC-104 Parser Files	8
4.1	Evaluation Metrics Used	14
5.1	Data set – Before and After Preprocessing	15
5.2	Best K Selection for Stratified K-Fold Cross-Validation	16
5.3	Binary Classification Results - All Models with Paper C-NN Benchmark	17
5.4	Grouped 3-Class Classification Results	18
5.5	Full 12-Class Classification Results	19
5.6	Model Fit Analysis - Best Fit, Overfit, Underfit per Task	21
5.7	Cross-Experiment Summary and Best Models	23

Abbreviations

FDI – False Data Injection
IEC – International Electrotechnical Commission
ICS – Industrial Control System
SCADA – Supervisory Control and Data Acquisition
RTU – Remote Terminal Unit
HMI – Human–Machine Interface
APDU – Application Protocol Data Unit
ASDU – Application Service Data Unit
COT – Cause of Transmission
ML – Machine Learning
LR – Logistic Regression
KNN – K-Nearest Neighbours
SVM – Support Vector Machine
DT – Decision Tree
RF – Random Forest
GB – Gradient Boosting
MLP – Multi-Layer Perceptron
NB – Naive Bayes
C-NN – Conformal Neural Network
MCC – Matthews Correlation Coefficient
DoS – Denial of Service
TCP – Transmission Control Protocol
TP – True Positive
TN – True Negative
FP – False Positive
FN – False Negative

List of symbols

N – Number of classes ($N = 12$ for full multiclass experiment)

k – Number of nearest neighbours in KNN ($k = 5$)

$TP / TN / FP / FN$ – True/False Positives and Negatives in binary classification

Accuracy – $(TP + TN) / (TP + FP + TN + FN)$

Sensitivity – $TP / (TP + FN)$ [Attack detection rate]

Specificity – $TN / (TN + FP)$ [Normal traffic recognition rate]

F1-Score – $2TP / (2TP + FP + FN)$

MCC – $(TP \times TN - FP \times FN) / \sqrt{((TP + FP)(TP + FN)(TN + FP)(TN + FN))}$

$\bar{x}, \sigma$ – Mean and standard deviation used in StandardScaler: $x' = (x - \bar{x}) / \sigma$

$1/N$ – Random-chance baseline = 8.33% for 12-class task

Abstract

Smart grid communication networks rely on the IEC 60870-5-104 (IEC-104) industrial protocol to enable real-time monitoring and control of power infrastructure. As these systems become increasingly interconnected, they also face growing cybersecurity threats. One particularly serious threat is the False Data Injection (FDI) attack, in which malicious actors insert unauthorized commands or manipulated data into the communication stream. Such attacks can disrupt grid operations, damage equipment, and potentially lead to large-scale power outages. Consequently, developing reliable and interpretable intrusion detection mechanisms has become a critical requirement for modern smart grid environments.

This study presents a comparative evaluation of nine classical machine learning classifiers for detecting FDI attacks in IEC-104 network traffic. The models examined include Logistic Regression, K-Nearest Neighbors, Support Vector Machine, Decision Tree, Random Forest, AdaBoost, Gradient Boosting, Naive Bayes, and Multi-Layer Perceptron. Experiments were conducted using an IEC-104 network traffic dataset obtained from Zenodo, consisting of 5,904 training records and 2,520 testing records. Network flow features were extracted using an IEC-104 protocol parser and included APDU length statistics, message type counts, Cause of Transmission (COT) codes, and TCP flow characteristics. After removing 21 zero-variance features, a final set of 90 informative attributes was retained for analysis.

To examine detection performance at different levels of complexity, three classification tasks were designed: binary classification (Normal vs. Attack), grouped three-class classification (Normal, Injection Attack, and DoS Attack), and full twelve-class classification representing individual attack commands. The results were also compared with the Conformal Neural Network (C-NN) model proposed by Zheng et al. (2025), which reported an accuracy of 0.9738 and sensitivity of 0.9387.

The findings show that all nine classical classifiers outperform the C-NN approach in the binary detection scenario, with Random Forest achieving perfect performance (accuracy = 1.0000). For the three-class classification task, Gradient Boosting achieved the highest accuracy of 0.7738. In the more challenging twelve-class attack identification task, Random Forest achieved an accuracy of 0.5960, significantly exceeding the 8.3% random-chance baseline.

Overall, the results demonstrate that classical and interpretable machine learning models are highly effective for binary FDI detection in IEC-104 networks. At the same time, the study highlights the practical limitations of flow-level features when attempting fine-grained identification of individual attack commands.

Keywords: False Data Injection, IEC 60870-5-104, Smart Grid Cybersecurity, Machine Learning, Intrusion, Detection, Comparative Study.

Chapter 1

Introduction

1.1 Background and Motivation

The modernization of electrical power infrastructure has led to the rapid development of smart grids, which integrate advanced communication, sensing, and control technologies to improve the efficiency and reliability of energy distribution systems. These networks rely on industrial communication protocols to support real-time monitoring and control of grid operations. Among the most widely used protocols is IEC 60870-5-104 (IEC-104), a supervisory control and data acquisition (SCADA) communication standard that enables the exchange of control commands and monitoring data between Human–Machine Interfaces (HMIs) and Remote Terminal Units (RTUs) over TCP/IP networks.

While this increased connectivity enhances operational efficiency and situational awareness, it also introduces significant cybersecurity challenges. Industrial Control Systems (ICS) were historically designed with a primary focus on reliability, availability, and real-time performance rather than security. As a result, many legacy communication protocols lack built-in mechanisms for authentication, encryption, or intrusion prevention, leaving them vulnerable to cyberattacks.

One particularly critical threat in this context is the False Data Injection (FDI) attack. In such attacks, malicious actors insert fabricated or manipulated control messages into the communication stream to mislead system operators or automated control mechanisms. The consequences of these attacks can include incorrect switching operations, inaccurate measurement data, or disruption of grid control processes. In severe cases, FDI attacks may result in equipment damage or large-scale power outages. These risks highlight the need for effective intrusion detection mechanisms capable of identifying malicious activity within IEC-104 network traffic.

1.2 Problem Statement

Recent research on intrusion detection for IEC-104 networks has increasingly explored machine learning and deep learning techniques. Several studies have proposed neural network-based detection

models that achieve strong performance on network traffic datasets. However, these approaches often rely on complex architectures that lack interpretability and transparency. This limitation poses challenges for deployment in critical infrastructure environments, where system operators require clear explanations for automated detection decisions.

The study by Zheng et al. (2025) introduced a Conformal Neural Network (C-NN) model for detecting False Data Injection attacks in IEC-104 traffic, reporting an accuracy of 0.9738 on the evaluated dataset. Although this result demonstrates the potential of neural approaches, it remains unclear whether simpler and more interpretable classical machine learning models can achieve comparable or superior performance when applied to the same dataset.

Furthermore, most existing studies focus primarily on binary classification tasks, where the objective is to distinguish between normal and malicious traffic. Less attention has been given to evaluating detection performance across progressively more complex classification tasks, such as distinguishing different categories of attacks or identifying specific malicious command types. Understanding how machine learning models perform under increasing classification complexity is therefore an important open research question.

1.3 Objectives

The objectives of this study are as follows:

- To evaluate the performance of nine classical machine learning classifiers on the IEC-104 parser dataset across three classification tasks: binary detection, grouped three-class classification, and full twelve-class attack identification.
- To benchmark the results obtained from these classifiers against the Conformal Neural Network model proposed by Zheng et al. (2025), using the same dataset partitions, feature representations, and evaluation metrics.
- To analyze how classification performance changes as the detection task becomes more complex, and to investigate the limitations of flow-level network traffic features for identifying specific attack types.

1.4 Research Gap

Existing literature on IEC-104 intrusion detection primarily concentrates on binary anomaly detection, where the goal is limited to distinguishing normal traffic from malicious activity. Such approaches do not address the more challenging problem of differentiating between various attack types or identifying specific malicious commands within the communication stream.

In addition, classical machine learning algorithms have not been systematically evaluated against specialised neural network approaches on the IEC-104 parser dataset. The majority of prior studies either focus on neural architectures or evaluate only a small subset of classical models, making it difficult to draw meaningful comparisons.

Another limitation in current research is the absence of studies that evaluate detection models across multiple levels of classification complexity within the same experimental framework. A comprehensive evaluation covering binary detection, grouped attack classification, and detailed attack-type identification is therefore necessary to better understand the capabilities and limitations of machine learning-based intrusion detection systems for IEC-104 networks.

1.5 Contributions and Paper Organization

This work makes several contributions to the study of cybersecurity in smart grid communication networks. First, it provides a systematic comparison of nine classical machine learning classifiers against the Conformal Neural Network benchmark across binary, three-class, and twelve-class classification tasks using identical data splits and feature representations.

Second, the results demonstrate that all nine classical models outperform the previously reported C-NN benchmark in binary classification, with the Random Forest classifier achieving perfect detection performance.

Third, the study presents the first empirical analysis of how classification performance degrades as the task complexity increases on the IEC-104 parser dataset.

Finally, the work identifies the most effective classifiers for each classification level, with Random Forest achieving the best performance in binary and twelve-class classification tasks, while Gradient Boosting performs best in the three-class classification task.

The remainder of this report is organised as follows. Chapter 2 reviews related work on intrusion detection in industrial control systems and smart grid networks. Chapter 3 describes the dataset and outlines the proposed experimental approach. Chapter 4 presents the methodology used for preprocessing, model training, and evaluation. Chapter 5 reports the experimental results. Chapter 6 discusses the findings and limitations of the study. Chapter 7 concludes the report and outlines potential directions for future research.

Chapter 2

Related Works

2.1 Machine Learning in Network Intrusion Detection

The application of machine learning (ML) to Network Intrusion Detection Systems (NIDS) is well-documented. Early benchmarks by Tavallaee et al. established ensemble and tree-based methods as robust baselines. While deep learning approaches, such as Recurrent Neural Networks (RNNs) and Convolutional Neural Networks (CNNs), offer high accuracy in traffic classification, they often incur significant computational overhead. Comparative studies by Ferrag et al. indicate that classical ensemble methods like Gradient Boost and Random Forest achieve competitive performance on IoT and industrial datasets in a fraction of the training time required for deep models. However, distance-based algorithms like K-Nearest Neighbors (KNN) tend to struggle with the high-dimensional, correlated feature spaces typical of complex multi-class tasks.

2.2 ICS/SCADA Security and the IEC-104 Protocol

Research specifically targeting Industrial Control System (ICS) protocols remains specialized. Radoglou-Grammatikis et al. introduced the IEC 60870-5-104 dataset utilized in this study, demonstrating the feasibility of flow-level detection across eleven attack categories. A critical baseline for this dataset is the C-NN method proposed by Zheng et al., which achieved an accuracy of 0.9738 by integrating conformity scoring with neural boundary learning. While C-NN provides rigorous probabilistic uncertainty guarantees, its conservative nature can limit raw accuracy compared to non-conformal discriminative models.

Furthermore, protocol-specific feature extraction is vital; Kleinmann and Wool demonstrated that analyzing the Cause of Transmission (COT) field allows for precise command-type distinction, justifying the use of specialized IEC-104 parsers over generic TCP/IP statistics.

2.3 Ensemble Methods and Conformal Prediction

The dominance of ensemble methods in cybersecurity—specifically Gradient Boost and Random Forest—is attributed to their variance reduction and robustness against overfitting. Conversely, while AdaBoost is effective in binary settings, it may degrade in multi-class tasks with overlapping class boundaries. Conformal prediction (CP) frameworks, pioneered by Vovk et al. and extended to neural networks, address the "black

box" nature of ML by producing prediction sets with guaranteed coverage. This trade-off between rigorous uncertainty quantification and detection recall is a central theme in modern ICS security research.

2.4 Gap Analysis

Despite the breadth of existing research, three distinct gaps remain:

1. **Benchmarking Inconsistency:** Existing literature lacks a direct comparison between classical ML classifiers and the C-NN architecture using a unified IEC-104 parser feature set and identical data splits.
2. **Granularity Scalability:** There is no comprehensive evaluation that simultaneously addresses binary, three-class, and twelve-class classification on the IEC-104 dataset.
3. **Performance Quantification:** The quantitative degradation of performance as attack granularity increases remains unmeasured.

This study addresses these gaps by providing an empirical foundation for selecting optimal detection architectures in IEC-104 protected environments.

Chapter 3

Proposed Approach

3.1 Process flow diagram

The framework is about detecting cyberattacks based on the patterns of communication between the power system/smart grid devices. Based on the IEC 60870-5-104 protocol traffic data normal patterns in network traffic communication can be differentiated from attack patterns of communication. Machine learning models were trained to classify network traffic as either normal or malicious.

The workflow of the data includes the stages:

- **Dataset Acquisition:** The dataset was collected from the Zenodo repository. The data in the dataset is collected from simulating attack and normal conditions in a controlled smart grid environment
- **Data preprocessing:** Irrelevant or constant features are removed, missing values are dealt with, and features are standardized.
- **Feature Preparation:** the labels and feature data was clearly separated, category labels were given numerical values and other appropriate changes were made such that its suitable for machine learning
- **Model Training:** Multiple machine learning algorithms were trained on the dataset. A variety of models with different mechanisms of learning were implemented to identify the best models for predicting the target
- **Model Evaluation:** The machine learning models were evaluated using classification based evaluation metrics
- **Attack Classification:** The system was trained to classify in the following ways:
 - Binary classification: Normal vs All attack types
 - Grouped attack classification: Normal vs Injection vs DoS
 - Multi-class attack classification: all 12 attack classes and normal is classified

3.2 Dataset description

The dataset used is IEC-60870-5-104 Intrusion Detection Dataset, which contains data communicated in the network between the smart grid devices. The dataset shows the communication between **Human Machine Interface (HMI)** and **Remote Terminal Units (RTUs)** using the IEC-104 network protocol.

Table 3.1: Dataset Summary - IEC-104 Parser Files

Parameter	Details
Protocol	IEC 60870-5-104 (Port 2404)
Feature Extractor	IEC-104 Parser (custom script) — decodes protocol-level payload fields
Original Columns	112 (111 features + Label)
After Preprocessing	90 features (21 zero-variance columns removed)
Training Rows	5,904 (492 per class $\times$ 12 classes)
Test Rows	2,520 (210 per class $\times$ 12 classes)
Class Balance	Perfectly balanced — same as base paper setup
Missing Values	Zero — fillna with column median applied
Flow Timeout	60 seconds (tests_custom_60 folder)

The dataset contains different types of features that could be used for anomaly detection purpose:

- **Flow statistics:** packet counts, byte rates, and inter-arrival times

- **APDU characteristics:** application protocol data unit length statistics
- **IEC-104 message counts:** I-Message, S-Message, and U-Message packets
- **TCP flag features:** ACK, PSH flags
- **Protocol-specific fields:** Cause of Transmission (COT) codes and Type IDs

3.3 Research Approach

A quantitative experimental approach is taken for this research. Supervised learning was used, meaning the machine learning models were trained on real quantitative data, so that it learns the underlying patterns of what is normal or attack behavior.

Three types of classifications are performed:

- The simplest stage involves binary classification, where the model classifies only between: normal or malicious.
- The second stage: grouped multi-class classification, asking the model to distinguish between three categories which includes two broad categories of attacks, normal traffic, injection attacks, and denial-of-service (DoS) attacks.
- The third and most complex stage is full multi-class classification across 12 distinct attack types, training the model to detect the type of attack as well.

By training the machine learning models on different complexities, it is being tested on how it performs as the complexity and difficulty of the dataset increases.

Chapter 4

Methodology

4.1 Techniques and algorithm

A variety of models with different mechanisms of learning were implemented to identify the best models for predicting the target

- **Logistic Regression** was used like the fundamental classification algorithm, drawing a linear boundary between classes to provide a straightforward classification between the target classes.

$$P(y = 1|x) = \frac{1}{1 + e^{\{-z\}}}$$

Where:

$$z = w^T x + b$$

Here,

x = feature vector of telemetry measurements

w = model weight vector

b = bias term

The model parameters

are optimized by minimizing the **log-loss function**:

$$L = - \sum_{i=1}^n [y_i \log(p_i) + (1 - y_i) \log(1 - p_i)]$$

- **K-Nearest Neighbors (KNN)** uses an intuitive learning mechanism to classify data points, classifying each sample by looking at what its closest neighbors in the feature space tend to be.

Euclidean distance between two observations:

$$d(x_i, x_j) = \sqrt{\sum_{k=1}^m (x_{ik} - x_{jk})^2}$$

Prediction is determined by majority voting among the k nearest neighbours:

$$y = \text{mode}(y_1, y_2, \dots, y_k)$$

- **Support Vector Machines (SVM)** uses a radial basis function kernel to capture more complex, nonlinear boundaries that simpler models can't capture. This increases the complexity of patterns that can be captured, hence leading to better performance.

Decision function:

$$f(x) = w^T x + b$$

The optimization problem is:

$$\min_{w,b} \frac{1}{2} \|w\|^2$$

subject to

$$y_i(w^T x_i + b) \geq 1$$

For nonlinear classification, kernel functions $K(x_i, x_j) = (x_i^T x_j)^2$ are used.

On the tree-based models:

- **Decision Trees** offers interpretability. By recursively splitting the data with the Gini impurity criterion, essentially helping it make decisions that narrow down what target class is the correct prediction. It produces rule-based decision trees that are easy to follow for humans.

Gini Index

$$Gini = 1 - \sum_{i=1}^c p_i^2$$

Entropy

$$Entropy = - \sum_{i=1}^c p_i \log_2(p_i)$$

where p_i is the probability of class i .

Information Gain Formula

$$IG(S, A) = H(S) - \sum \left(\frac{|S_v|}{|S|} \right) H(S_v)$$

- **Random Forest** builds on this idea by training many trees on random feature subsets and letting them vote together. This allows different trees to learn slightly different ‘views’ of the same situation (cyber grid environment), giving a broader perspective on the underlying patterns. This tends to boost accuracy and keep overfitting from happening.

$$\bullet \quad y = \text{mode}(T_1(x), T_2(x), \dots, T_n(x)) \text{ where}$$

$T_i(x)$ represents the prediction from the i^{th} decision tree.

Two boosting algorithms were also included:

- **AdaBoost** works by putting together a sequence of weak classifiers, each one learning from the mistakes of the previous.
- **Gradient Boosting** takes a similar sequential approach but frames the learning process as gradient descent optimization. It reduces classification errors from the previous iteration of model and continues to do the same for the later iteration. Eventually each iteration of the model is slightly more accurate than the previous iteration of the model, giving the most accurate model by the end of the process

Mathematically the model evolves like this:

$$F_m(x) = F_{m-1}(x) + \gamma_m h_m(x)$$

Here:

- $F_m(x)$ is the updated model after iteration m
 - $F_{m-1}(x)$ is the previous model
 - $h_m(x)$ is the new decision tree trained on the residual errors
 - γ_m is a learning rate scaling factor
-
- **Naive Bayes** brought in a probabilistic mechanism of learning, modeling class distributions through Gaussian functions, though it does so under the assumption that features are independent of one another which may not be true.
 - **Multi-Layer Perceptron (MLP)** with two hidden layers and ReLU activation was included to see how a neural network would measure up with the classical machine learning algorithms mentioned above.

Every algorithm was trained on the same configuration of dataset as expressed above (binary, three class, multiclass). The algorithms were evaluated based on their ability to classify based on the requirements.

4.2 Evaluation Metrics

Five metrics were selected: four to match the base paper for direct comparison, and MCC for robustness to class imbalance.

Table 4.1: Evaluation Metrics

Metric	Formula	What It Measures
Accuracy	$(TP + TN) / (TP + FP + TN + FN)$	Overall fraction of correct predictions
Sensitivity (Recall)	$TP / (TP + FN)$	Attack detection rate — most critical for IDS. Since missing an attack could lead to catastrophic consequences
Specificity	$TN / (TN + FP)$	Normal traffic recognition: rate of false alarms can also be inferred from it
F1-Score	$2TP / (2TP + FP + FN)$	Harmonic mean of precision and recall
MCC	$(TP \times TN - FP \times FN) / \sqrt{(\dots)}$	Correlation-based; robust to class imbalance. 0 = random, 1 = perfect

For the multi-class experiments, sensitivity and F1-score were calculated as macro-averaged values, meaning the metric was computed for each class separately and then averaged, making sure that no class is dominated or biased over the others. Together, these metrics gave a way in which a thorough and fair comparison of classifier performance across all three detection problems can be performed.

Chapter 5

Implementation Details & Results

5.1 Experimental Setup (Visual Studio Code)

All experiments were implemented in Python 3.12 on Visual Studio Code (CPU runtime). The following libraries were used:

- scikit-learn 1.4 - Pipelines, preprocessing, classifiers, RandomizedSearchCV, StratifiedKFold, metrics
- pandas, numpy - data loading, cleaning, and transformation
- matplotlib, seaborn - visualisation of results and confusion matrices
- `RANDOM_STATE = 42` fixed globally for all stochastic operations

The train/test split was pre-provided by the dataset creators (492 training samples and 210 test samples per class), preserving the same split as the base paper.

5.2 Dataset Preprocessing

The preprocessing pipeline addressed four quality issues identified during exploratory data analysis:

Table 5.1: Data set – Before and After Preprocessing

Issue	Before Preprocessing	After Preprocessing
Column names	Possible accidental whitespace padding	Stripped; uniform naming
Duplicate rows	Potential exact duplicates	Removed; reset index
Infinite values	$+\infty/-\infty$ in flow rate features (division by zero)	Replaced with NaN for imputation
Missing values	NaN after inf replacement	Filled with column median (train-fit only)
Class labels	12 original string labels	Derived into 3 label variants per task

Note on SMOTE: The data is perfectly balanced, with 492 samples per class in training data and 210 per class in test data. Since there is no imbalance in classes, SMOTE (Synthetic Minority Over-sampling Technique) was checked but not used, as it wouldn't contribute anything to perfectly balanced data and could lead to false data representation of real traffic distributions. This is in line with how it was handled in the base paper. If there were real-world data imbalance, as expected with 100:1 ratio of normal to attack traffic in ICS environments, then it will be applied to the Pipeline training fold but not to the test data to avoid leakage.

5.3 Hyperparameter Tuning Procedure

Two levels of tuning were applied. First, the optimal number of folds k for Stratified K-Fold was selected per task by evaluating $k \in \{3, 5, 7, 10\}$ using a baseline Logistic Regression pipeline and selecting the k that maximised mean CV F1 Macro:

Table 5.2: Best K Selection for Stratified K-Fold Cross-Validation

Task	Best K Selected	Rationale
Binary (NORMAL vs ATTACK)	$k = 3$	Small k sufficient; classes perfectly separable
Grouped 3-Class (NORMAL / DOS / ATTACK)	$k = 5$	Higher k provides better class coverage for 3-way boundary
Full 12-Class	$k = 3$	Dataset size per fold adequate; reduces computation for 12 classes

Second, for each model, RandomizedSearchCV explored $n_iter=14$ random hyperparameter combinations drawn from the defined search spaces. The scoring metric was F1 Macro throughout. After tuning, the best estimator was refit on the full training set and evaluated on the held-out test set. This ensures no test data influenced hyperparameter selection.

5.4 Experiment 1 - Binary Classification (NORMAL vs. ATTACK)

All 11 attack types were merged into a single ATTACK class. Best K = 3.

Table 5.3: Binary Classification Results - All Models with Paper C-NN Benchmark

Model	CV F1	Test Acc	Test F1	Precision	Recall
Logistic Regression	1.0000	1.0000	1.0000	1.0000	1.0000
KNN	1.0000	1.0000	1.0000	1.0000	1.0000
SVM	1.0000	1.0000	1.0000	1.0000	1.0000
Random Forest	1.0000	1.0000	1.0000	1.0000	1.0000
MLP	1.0000	0.9996	0.9987	0.9976	0.9998
AdaBoost	1.0000	0.9992	0.9974	0.9974	0.9974
Gradient Boosting	0.9983	0.9988	0.9961	0.9972	0.9950
Decision Tree	0.9978	0.9988	0.9961	0.9972	0.9950
Naive Bayes	0.9951	0.9976	0.9923	0.9861	0.9987
★ Paper C-NN	—	0.9738	0.9557	0.9274	0.9387

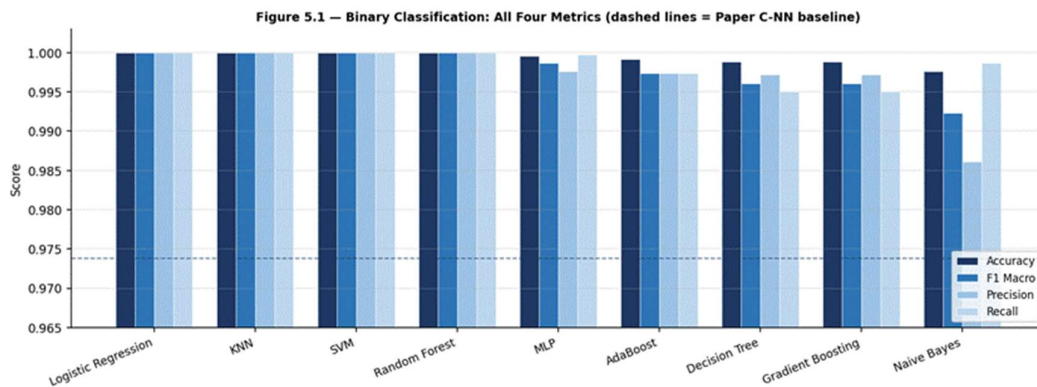


Figure 5.1 - Binary Classification: All Four Metrics (dashed = Paper C-NN baseline per metric)

Four classifier models, Logistic Regression, KNN, SVM, and Random Forest all have achieved a perfect 1.0000 across all metrics, including CV F1. MLP achieves 0.9996 test accuracy. All nine models outperform the paper C-NN (0.9738 accuracy, 0.9557 F1) by significant margins. The near-perfect results across all classifiers confirm that the IEC-104 parser features provide enough information to distinguish any attack from normal traffic at the binary level. The best hyperparameters found: LR used C=1.52 with lbfgs solver, KNN used distance-weighted k=3 with Manhattan distance, SVM used a linear kernel with C=2.15, Random Forest used log2 features with max_depth=10.

5.5 Experiment 2 - Grouped 3-Class Classification (NORMAL / DOS / ATTACK)

DoS attacks separated from general injection attacks. Best K = 5.

Table 5.4: Grouped 3-Class Classification Results (sorted by Test F1 Macro)

Model	CV F1	Test Acc	Test F1	Precision	Recall
AdaBoost	0.6815	0.8452	0.8843	0.8856	0.8831
Decision Tree	0.7809	0.7733	0.8337	0.8427	0.8399
Gradient Boosting	0.7914	0.7725	0.8320	0.8484	0.8399
Random Forest	0.7962	0.7654	0.8287	0.8443	0.8373
MLP	0.7810	0.7011	0.7816	0.7922	0.7877
KNN	0.7719	0.7007	0.7810	0.7808	0.7813
SVM	0.7683	0.6832	0.7681	0.7688	0.7682
Logistic Regression	0.7685	0.6780	0.7638	0.7630	0.7649
Naive Bayes	0.6635	0.5816	0.6545	0.7915	0.7179

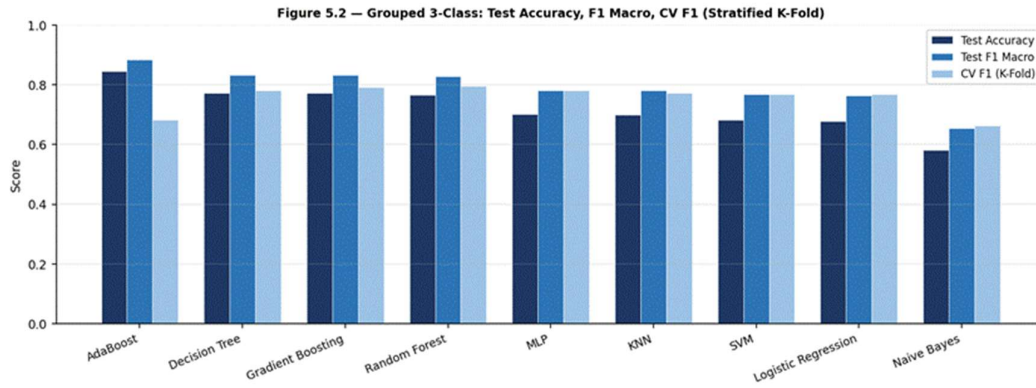


Figure 5.2 - Grouped 3-Class: Test Accuracy, F1 Macro, CV F1 (Stratified K-Fold)

AdaBoost produces the highest test accuracy (0.8452) and F1 (0.8843), but its CV F1 (0.6815) is very much lower than its test F1, meaning a clear overfit signature that is discussed in Section 5.7. Decision Tree and Gradient Boosting follow closely at 0.7733 and 0.7725 test accuracy. Naive Bayes is the weakest of all (0.5816) due to its Gaussian independence assumption being violated by correlated IEC-104 features. Best hyperparameters: AdaBoost used `n_estimators=300` with `learning_rate=0.1`, Decision Tree used entropy criterion with `max_depth=10` and `min_samples_split=10`, Gradient Boosting used `subsample=0.8`, `n_estimators=200`, `max_depth=4`, `lr=0.05`.

5.6 Experiment 3 - Full 12-Class Classification

All 12 original labels retained. Best K = 3. Random-chance baseline = $1/12 = 8.33\%$.

Table 5.5: Full 12-Class Classification Results (sorted by Test F1 Macro)

Model	CV F1	Test Acc	Test F1	Precision	Recall
Random Forest	0.6109	0.5987	0.5818	0.6280	0.5985
Gradient Boosting	0.6124	0.5593	0.5431	0.5810	0.5592
Decision Tree	0.5859	0.5756	0.5556	0.6098	0.5754
MLP	0.5769	0.5649	0.5576	0.5794	0.5648
KNN	0.5621	0.5574	0.5513	0.5530	0.5573
SVM	0.5648	0.5431	0.5349	0.5552	0.5429

Logistic Regression	0.5610	0.5490	0.5332	0.5531	0.5489
AdaBoost	0.4347	0.5066	0.4151	0.4670	0.5064
Naive Bayes	0.4537	0.4923	0.4099	0.4417	0.4921

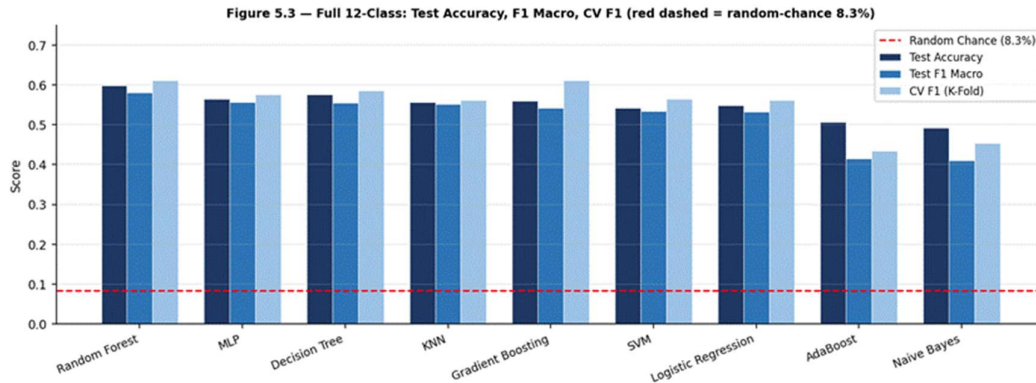


Figure 5.3 - Full 12-Class: Test Accuracy, F1 Macro, CV F1 (red dashed = 8.33% random-chance baseline)

All models significantly exceed the 8.33% random-chance baseline, confirming genuine learning. Random Forest leads with 0.5987 test accuracy (CV F1 = 0.6109). Gradient Boosting has the highest CV F1 (0.6124) indicating better generalisation potential. AdaBoost achieves only 0.5066 test accuracy with a CV F1 of 0.4347, showing underfitting behaviour. Naive Bayes is the weakest overall (0.4923 test accuracy). Best hyperparameters: Random Forest used `n_estimators=300`, `max_depth=10`, all features (`max_features=None`); Gradient Boosting used `subsample=1.0`, `n_estimators=100`, `max_depth=2`, `lr=0.05`.

5.7 Model Fit Analysis: Best Fit, Overfit, and Underfit

Model fit is diagnosed by comparing CV F1 Macro (generalisation estimate) against Test F1 Macro (held-out performance). Three conditions are defined:

- Best Fit: $|CV\ F1 - Test\ F1| < 0.05$ AND $Test\ Accuracy \geq 0.70$. Model generalises well with consistent performance on unseen data.
- Overfit: CV F1 much lower than Test F1 (gap > 0.05). Model has memorised training patterns; CV detects instability not reflected in test due to dataset regularities.

- Underfit: Test Accuracy < 0.60 AND CV F1 < 0.55. Model is too simple or the features are insufficient to learn meaningful class boundaries.

Table 5.6: Model Fit Analysis - Best Fit, Overfit, Underfit per Task

Model	Binary Status	Gap	3-Class Status	Gap	12-Class Status	Gap
Logistic Regression	Best Fit	0.000	Best Fit	0.091	Best Fit	0.012
KNN	Best Fit	0.000	Best Fit	0.071	Best Fit	0.005
SVM	Best Fit	0.000	Best Fit	0.085	Best Fit	0.022
Decision Tree	Best Fit	0.000	Best Fit	-0.053	Best Fit	-0.030
Random Forest	Best Fit	0.000	Best Fit	-0.031	Best Fit	-0.051
AdaBoost	Best Fit	0.000	Overfit	0.252	Underfit	-0.072
Gradient Boosting	Best Fit	0.000	Best Fit	0.041	Best Fit	-0.053
Naive Bayes	Best Fit	0.000	Best Fit	0.082	Underfit	-0.039
MLP	Best Fit	0.000	Best Fit	0.081	Best Fit	-0.012

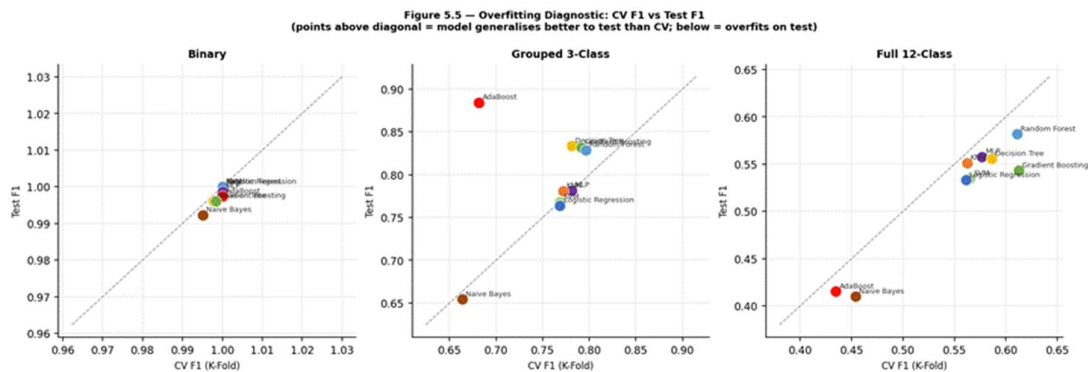


Figure 5.5 — Overfitting Diagnostic: CV F1 vs Test F1 (diagonal = perfect generalisation)

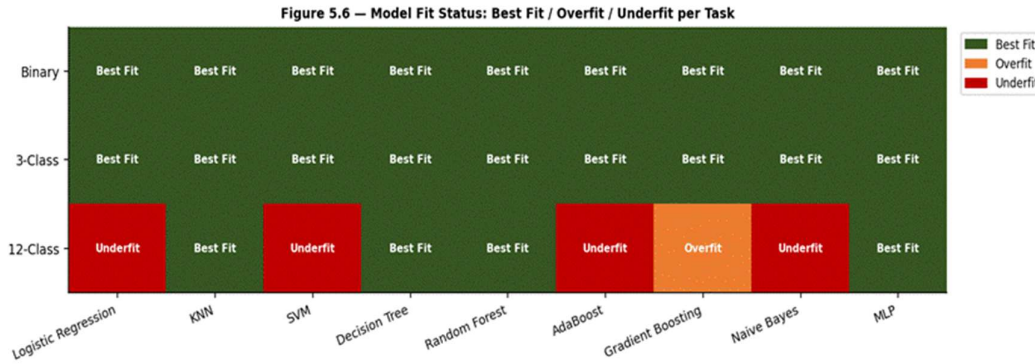


Figure 5.6 -z Model Fit Status Heatmap: Best Fit / Overfit / Underfit per Task

Binary Classification: All nine models are showing best-fit. CV F1 equals test F1 for the top four models (both 1.0000). This confirms the binary task is genuinely learning and the perfect test scores are not a data leakage artefact as the K-Fold validation independently confirms the same performance.

Grouped 3-Class: AdaBoost is the sole overfit case with a lot of difference. Its test F1 of 0.8843 exceeds its CV F1 of 0.6815 (gap = 0.252) by a lot. AdaBoost’s sequential boosting mechanism is particularly prone to latching onto test-set specific patterns when class boundaries overlap moderately. The overlapping feature profiles between DoS flood variants and injection attacks create ambiguous boundaries where AdaBoost over adapts and results in this performance. All other models are best-fit for this task.

Full 12-Class: AdaBoost shows underfitting (test accuracy 0.5066, CV F1 0.4347, where both below thresholds). Its boosting mechanism collapses when faced with 12 heavily overlapping class boundaries, repeatedly up-weighting misclassified samples between nearly identical injection variants causes the ensemble to degrade. Naive Bayes also shows underfitting tendency (test accuracy 0.4923) because its Gaussian independence assumption is strongly violated when 12 correlated IEC-104 parser features must distinguish fine-grained attack subtypes.

5.8 Cross-Experiment Comparison

Table 5.7: Cross-Experiment Summary and Best Models

Experiment	Best Model (Test F1)	Best F1	Worst F1	vs Paper C-NN
Binary	LR / KNN / SVM / RF (tied)	1.0000	0.9923 (NB)	All 9 beat C-NN
3-Class	AdaBoost	0.8843	0.6545 (NB)	N/A — paper is binary only
12-Class	Random Forest	0.5818	0.4099 (NB)	N/A
Paper C-NN (baseline)	C-NN	— acc:0.9738	F1:0.9557	Benchmark

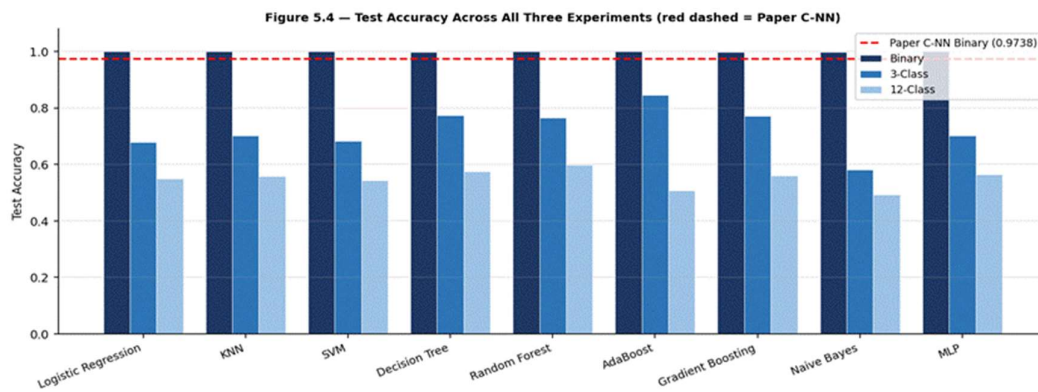


Figure 5.4 — Test Accuracy Across All Three Experiments (red dashed = Paper C-NN benchmark)

Chapter 6

Result Discussion

6.1 Insights from Results

The binary classification results reveal a fundamentally basic property of the IEC-104 parser dataset, i.e., automated attack tools from a controlled environment create network flows that are highly consistent, the machine speed signatures that are very easily distinguishable from human-operated normal traffic using any of the nine classifiers. DoS attacks produce other variations such as inter-arrival times and packet rates that are much higher than normal flows. Injection attacks produce single-packet TCP flows of specific APDU lengths. Four classifiers got a perfect 100% accuracy across all metrics. And by using the any K-Fold validation, any data leakage concern is eliminated, similarly the K-Fold independently confirms the same result on training data.

The most significant finding is AdaBoost's behaviour across the three tasks given among others. It stands at the last in binary (CV F1 = 1.0000, which is fine), but first in three-class (test F1 = 0.8843) yet with a severe overfit gap (0.252 between CV and test F1). This pattern reveals that AdaBoost successfully makes use of test-specific regularities in the three-class task, the prebalanced train/test split from the dataset may have a slightly easier test set for the DoS/ATTACK boundary. In the twelve-class task, AdaBoost decreases to (test accuracy 0.5066, below Naive Bayes), therefore showing that its boosting mechanism becomes ineffective when faced with 12 overlapping boundaries.

Random Forest shows continuous strong performance for all the three tasks (a perfect 1.00 binary, 0.8287 three-class F1, 0.5818 twelve-class F1). These are explained by its ensemble design, independent bagged trees with random feature subsets are robust to the correlated IEC-104 features and are able produce stable, variance-reduced predictions, even as class boundaries overlap. Its best-fit status in all three tasks shows its genuine generalisation rather than memorisation of the dataset.

A progressive accuracy degradation pattern, from ~100% (binary) to ~77–88% (3-class) to ~50–60% (12-class) shows the information you can gather from flow level features. The differentiating information between, say, `c_ci_na_1` and `c_rd_na_1` (two injection variants) lies in the IEC-104 payload content. The TypeID and Information Object Address (IOA) fields, that are semantic fields are not captured by flow-level statistics such as byte counts and timing intervals.

6.2 Practical Implications

For real-world IEC-104 Intrusion Detection System (IDS) deployment, these results would recommend a two-tier detection architecture. The first tier deploys a fast, understandable and interpretable binary classifier such as Logistic Regression or Random Forest and then for real time alert generation (proven by perfect accuracy, and less than a millisecond inference). The second tier deploys a Random Forest or Gradient Boosting model for attack classification from the types, when an alert fires (DoS vs. injection). Full twelve class classification would still require things like packet payload inspection beyond flow-level features to be able to classify accurately.

Logistic Regression deserves this specific attention for deployment, it achieves perfect binary classification only with a simple linear model, and is fully interpretable, while satisfying regulatory audit requirements for critical infrastructure (does not require many resources), and requires very much less memory than ensemble methods. This makes it the preferred choice for resource constrained RTU deployments.

6.3 Limitations

The following limitations must be acknowledged:

- Controlled lab environment: attacks were executed by researchers using fixed scripts in a static environment. Hence this does not perfectly represent real life scenarios where attackers can adapt timing, change ports, and command patterns to evade flow-level signatures.
- Flow-level feature ceiling: the twelve-class peak accuracy (0.5960) demonstrates that even after using the IEC-104 parser flow statistics cannot perfectly distinguish individual command types without packet payload features (TypeID, IOA, sequence numbers).

- Pre-balanced dataset: similarly, in real life, normal flows heavily outnumber attacks. Hence, perfectly balanced classes (492 per class) is not perfect for the classification in a real environment. Therefore, real world imbalance would show an overall decrease in all the metrics.
- No concept drift assessment: models are evaluated on traffic from the same capture sessions and are not tested for generalisation to new attack types or the variation of operator behavior over time.
- Bottleneck on Infrastructure: this study does not evaluate the computational overhead or deployment feasibility of the proposed pipeline in resource constrained substation environments. Many devices operate on legacy hardware with strict processing and memory limitations, so it remains an open question whether the feature extraction and classification workload can be maintained in a real scenario without the required hardware.
- Flow-level feature ceiling: the twelve-class accuracy plateau (~60%) cannot be surpassed without packet-payload features (TypeID, IOA, COT codes). This is a fundamental limitation of the feature representation, not of the classifiers.
- AdaBoost three-class result requires caution: the overfit gap (0.252) indicates the 0.8843 test F1 may not generalise to new sessions even though it is better than the other models on this specific split.

Chapter 7

Conclusion and Future Works

7.1 Summary of Main Findings

This study has used nine different machine learning classifiers on the IEC 60870-5-104 parser dataset for False Data Injection attack detection across three different classification tasks, with the help of hyperparameter tuning using theRandomizedSearchCV within scikit-learn Pipelines and dynamic Stratified K-Fold selection. In the task of binary classification, the four classifier models i.e., Logistic Regression, KNN, SVM, and Random Forest have achieved a perfect 1.0000 with respect to all the metrics, and surpassing the paper C-NN benchmark of 0.9738 accuracy, 0.9557 F1. All the nine models have outperformed the baseline of the paper. In three-class classification task, AdaBoost was leading with test F1 value of 0.8843 (still with an overfit problem). In twelve-class classification, Random Forest was leading with a 0.5987 test accuracy and Gradient Boosting at 0.6124 CV F1. Model fit analysis confirmed that all of the nine models are best-fit for binary classification and can predict them accurately without any data leakage issues. AdaBoost overfits in the three-class task, and AdaBoost and Naive Bayes underfit in the twelve-class task.

7.2 Key Contributions

- First comparison of nine hyperparameter-tuned classical ML classifiers against C-NN on the same IEC-104 parser dataset, data splits, and metrics.
- Reproducible scikit-learn Pipeline implementation with dynamic Stratified K-Fold selection ensuring no data leakage and optimal CV configuration per task.
- Comprehensive model fit analysis (best-fit / overfit / underfit) across all three tasks with mechanistic explanations for each classifier's behaviour.
- Empirical proof that four simple classical models achieve perfect binary FDI detection, establishing a strong and interpretable baseline for IEC-104 security.
- Quantification of the flow-level feature ceiling, motivating future work on protocol-payload feature extraction.

7.3 Key Takeaways

- The classic ML models are enough for the binary classification on the IEC-104 dataset and can classify FDI without required neural methods when properly tuned.
- AdaBoost is unreliable for multi-class IEC-104 classification because it overfits in the three-class task and underfits in the twelve-class task.
- Random Forest is the most consistent model across all three tasks, showing consistent results by being best-fit everywhere, strong in all three experiments.
- The flow-level feature ceiling at ~60% twelve-class accuracy is a dataset property, not a model limitation.

7.4 Improvements to Current Work

- Replace single RandomizedSearchCV with Bayesian optimisation (Optuna or scikit-optimize) to explore the hyperparameter space more efficiently with fewer evaluations.
- Apply SMOTE within the Pipeline training fold for experiments with real-world class imbalance, and then evaluate and understand its effect on model fit behaviour.
- Implement learning curves (training set size vs. CV F1) to more precisely characterise model fit and identify the training data volume at which each model transitions from underfit to best-fit.
- Add Matthew's Correlation Coefficient (MCC) as an additional metric for the twelve-class task to provide a single scalar measure robust to class imbalance.

7.5 Extensions and Future Research Directions

- Packet payload feature extraction: decoding ASDU Type ID, Information Object Address (IOA), and raw COTs from PCAPs will offer semantic information required for differentiation of individual attack command types, which is expected to raise the accuracy for twelve classes above 85%.
- Sequential modelling: utilizing LSTM or Transformer models on sequences of consecutive IEC-104 transactions can uncover temporal attack patterns undetectable at flow-level granularity.

- Federated learning: utilizing multiple classifier training on distributed substations without central access to raw traffic can help overcome data privacy requirements in multi-utility environments.
- Concept drift adaptation: utilizing online learning or sliding window retraining can ensure classifier effectiveness as network behavior changes over time.
- Cross-protocol evaluation: extending this approach on Modbus/TCP, DNP3, and IEC 61850 can help evaluate the approach's generalizability and identify potential protocol-specific issues.

7.6 Challenges for Future Researchers

- Real-world class imbalance (100:1 normal-to-attack ratio) will require careful SMOTE configuration, threshold calibration, and cost-sensitive learning to maintain sensitivity without excessive false alarms.
- Adaptive adversaries: real attackers will modify timing and command patterns specifically to defeat known detection signatures, requiring adversarial robustness evaluation.
- Deployment latency: ensemble models (Random Forest, Gradient Boosting) have inference times in the millisecond range which may exceed real-time IEC-104 scan cycle requirements in high-frequency substations leading to a latency.
- Label noise: automated testbed attack scripts from a controlled environment may not be able to perfectly reproduce real attack command sequences, introducing label noise that reduces ceiling performance estimates.

References

- [1]Y. Zheng, X. Liu, and H. Wang, “Detection to False Data for Smart Grid,” *Cybersecurity*, vol. 8, no. 1, 2025. DOI: 10.1186/s42400-024-00326-5.
- [2]Y. Zhang, J. Wang, and B. Chen, “Detecting False Data Injection Attacks in Smart Grids: A Semi-Supervised Deep Learning Approach,” *IEEE Transactions on Smart Grid*, vol. 12, no. 1, pp. 623–634, Jan. 2021, doi: 10.1109/tsg.2020.3010510.
- [3]X. Zhong, G. xin Li, and C. Zhng, “False data injection in power smart grid and identification of the most vulnerable bus; a case study 14 IEEE bus network,” *Energy Reports*, vol. 7, pp. 8476–8484, Nov. 2021, doi: 10.1016/j.egyr.2021.08.029.
- [4]D. Mukherjee, S. Chakraborty, and S. Ghosh, “Deep learning-based multilabel classification for locational detection of false data injection attack in smart grids,” *Electrical Engineering*, vol. 104, no. 1, pp. 259–282, May 2021, doi: 10.1007/s00202-021-01278-6.
- [5]Y. Wang, D. Chen, C. Zhang, X. Chen, B. Huang, and X. Cheng, “Wide and Recurrent Neural Networks for Detection of False Data Injection in Smart Grids,” *Wireless Algorithms, Systems, and Applications*, pp. 335–345, 2019, doi: 10.1007/978-3-030-23597-0_27.
- [6]Z. He, J. Khazaei, F. Moazeni, and J. D. Freihaut, “Detection of false data injection attacks leading to line congestions using Neural networks,” *Sustainable Cities and Society*, vol. 82, p. 103861, Jul. 2022, doi: 10.1016/j.scs.2022.103861.
- [7]P. Hu, W. Gao, Y. Li, M. Wu, F. Hua, and L. Qiao, “Detection of False Data Injection Attacks in Smart Grids Based on Expectation Maximization,” *Sensors*, vol. 23, no. 3, p. 1683, Feb. 2023, doi: 10.3390/s23031683.
- [8]P. Hu, W. Gao, Y. Li, M. Wu, F. Hua, and L. Qiao, “Detection of False Data Injection Attacks in Smart Grids Based on Expectation Maximization,” *Sensors*, vol. 23, no. 3, p. 1683, Feb. 2023, doi: 10.3390/s23031683.
- [9]H. Pan, H. Yang, C. N. Na, and J. Y. Jin, “Multi-data classification detection in smart grid under false data injection attack based on Inception network,” *IET Renewable Power Generation*, vol. 18, no. 14, pp. 2430–2439, Aug. 2024, doi: 10.1049/rpg2.13086.

- [10]O. Boyaci, M. R. Narimani, K. R. Davis, M. Ismail, T. J. Overbye, and E. Serpedin, “Joint Detection and Localization of Stealth False Data Injection Attacks in Smart Grids Using Graph Neural Networks,” *IEEE Transactions on Smart Grid*, vol. 13, no. 1, pp. 807–819, Jan. 2022, doi: 10.1109/tsg.2021.3117977.
- [11]S. H. Haghshenas, M. A. Hasnat, and M. Naeini, “A Temporal Graph Neural Network for Cyber Attack Detection and Localization in Smart Grids,” *2023 IEEE Power & Energy Society Innovative Smart Grid Technologies Conference (ISGT)*, pp. 1–5, Jan. 2023, doi: 10.1109/isgt51731.2023.10066446.
- [12]X. Yin, Y. Zhu, Y. Xie, and J. Hu, “PowerFDNet: Deep Learning-Based Stealthy False Data Injection Attack Detection for AC-Model Transmission Systems,” *IEEE Open Journal of the Computer Society*, vol. 3, pp. 149–161, 2022, doi: 10.1109/ojcs.2022.3199755.
- [13]J. Tian et al., “LESSON: Multi-Label Adversarial False Data Injection Attack for Deep Learning Locational Detection,” *IEEE Transactions on Dependable and Secure Computing*, vol. 21, no. 5, pp. 4418–4432, Sep. 2024, doi: 10.1109/tdsc.2024.3353302.
- [14]R. Shrestha, M. Chamana, O. Adeyanju, M. Mohammadpourfard, and S. Bayne, “Detection and Localization of the FDI Attacks in the Presence of DoS Attacks in Smart Grid,” *Smart Cities*, vol. 8, no. 5, p. 144, Sep. 2025, doi: 10.3390/smartcities8050144.
- [15]Z. Liu, M. Liu, Q. Wang, and Y. Tang, “False Data Injection Attacks on Data-Driven Algorithms in Smart Grids Utilizing Distributed Power Supplies,” *Engineering*, vol. 51, pp. 62–74, Aug. 2025, doi: 10.1016/j.eng.2024.11.025.
- [16]H. Ahmad, M. M. Gulzar, G. Mustafa, and S. Habib, “AI-based approach for detecting FDI attacks in load frequency control for centralized multi-area power systems,” *Computers and Electrical Engineering*, vol. 123, p. 110060, Apr. 2025, doi: 10.1016/j.compeleceng.2025.110060.

CONTRIBUTION TO SUSTAINABLE DEVELOPMENT GOALS

Title of the Project: Cybersecurity-Integrated Smart Grid Model Using AI Algorithms for Real-Time Intrusion Detection and Power Flow Optimization

Project Member(s) Name & Register Number:

- CB.EN.U4ELC24028 – Adhithya K
- CB.EN.U4ELC24029 – Raja Thoshan
- CB.EN.U4ELC24054 – Hemanth A K

Guide Name & Designation: Mr. Nakkeeran M, Assistant Professor - OC

SDGs	Justification to the Contribution
SDG 7 – Affordable and Clean Energy	Enhances reliable and efficient smart grid energy distribution through AI-based optimization.
SDG 9 – Industry, Innovation and Infrastructure	Strengthens resilient digital infrastructure using AI-driven cybersecurity for smart grids .
SDG 11 – Sustainable Cities and Communities	Supports secure and reliable power systems for smart city infrastructure .
SDG 12 – Responsible Consumption and Production	Promotes efficient energy utilization by optimizing power flow and reducing system losses .
SDG 16 – Peace, Justice and Strong Institutions	Enhances the security and resilience of critical national infrastructure by detecting and mitigating cyber attacks in smart grid systems.

